

Digital Signal Processing Lab

Digital Signal Processing

Getting Started with UniDAQ2

October 9, 2025

Hochschule für Angewandte Wissenschaften Hamburg
Hamburg University of Applied Sciences

© 2025 Copyright
Andrea Kupke, Prof. Dr.-Ing. Ulrich Sauvagerd, Prof. Dr.-Ing. Lutz Leutelt
Hochschule für Angewandte Wissenschaften Hamburg.

All rights reserved.

Alle Rechte, auch das des auszugsweisen Nachdrucks, der auszugsweisen oder vollständigen Wiedergabe, der Speicherung in Datenverarbeitungsanlagen und der Übersetzung, vorbehalten.

Dieses Dokument wurde mit Hilfe von KOMA-Script und LaTeX gesetzt.

Contents

1. Introduction **5**
1.1. Lab Hardware UniDAQ2 . . . 5

2. Start-up of the UniDAQ2 board and a first project **7**
2.1. Connecting the hardware . . . 7
2.2. Start of Code Composer Studio (CCS) and import of a project . . . 8
2.3. Compile, link and load the project . . . 9
2.4. Using the CCS Debugging tool . . . 9
2.4.1. Breakpoints . . . 10
2.4.2. Display source and assembler code . . . 10
2.4.3. Single step execution . . . 11
2.4.4. Monitor and change variables . . . 11
2.4.5. Show DSP register contents . . . 11
2.4.6. Show memory content . . . 12
2.4.7. Show contents of arrays graphically (Graphical Display) . . . 13
2.4.8. Code Profiling . . . 17
2.5. Signal input and output . . . 18
2.5.1. Signal input . . . 18
2.5.2. Signal output . . . 18

3. Sampling Rates **19**
3.1. Setting the sample rates . . . 19
3.2. Operation with different sampling rates on ADC and DAC . . . 19

4. Characteristics of the UniDAQ2 board **21**
4.1. Amplitude of the UniDAQ2 . . . 21
4.2. Anti-aliasing filtering . . . 21
4.2.1. Kemo Filter . . . 23

A. Appendix: UniDAQ2 **25**
A.1. Short presentation UniDAQ2 . . . 25
A.1.1. Typical fields of application . . . 26
A.1.2. Software . . . 26
A.1.3. Key Features . . . 26
A.2. Basic Sourcecode . . . 27



Chapter 1

Introduction

This tutorial describes the UniDAQ2 board and first steps with it. In chapter 2 it is first shown how to start up the board with a simple echo program. The possibilities and tools of the debugger described in section 2.4 can be looked up here during the further labs.

Furthermore, section 3 shows how to set the sampling rate and, for multirate systems, how to operate the demo program with different sampling frequencies at input and output (ADC/DAC).

The transfer characteristics of the UniDAQ2 board are discussed in detail in chapter 4. In particular, the non-constant amplitude response of the board versus frequency is pointed out and the usage of an external low-pass filter is shown.

1.1. Lab Hardware UniDAQ2

The UniDAQ2 board used in the Digital Signal Processing Lab (R.885) is based on the UniDAQ2 board from the company d.signT (for a short introduction see A.1) with additional own adapter boards.
Of the 16 ADCs and 8 DACs, 2 ADCs (ADC 0 and ADC 1) and 2 DACs (DAC 0 and DAC 1) are used.

Chapter 2

Start-up of the UniDAQ2 board and a first project

2.1. Connecting the hardware

The connection to the PC is done via USB using the JTAG emulator XDS200 (see fig. 2.2), which is connected to the JTAG connector of the board via an adapter.
For power consumption a 5V power supply is connected (for connections see fig. 2.1).

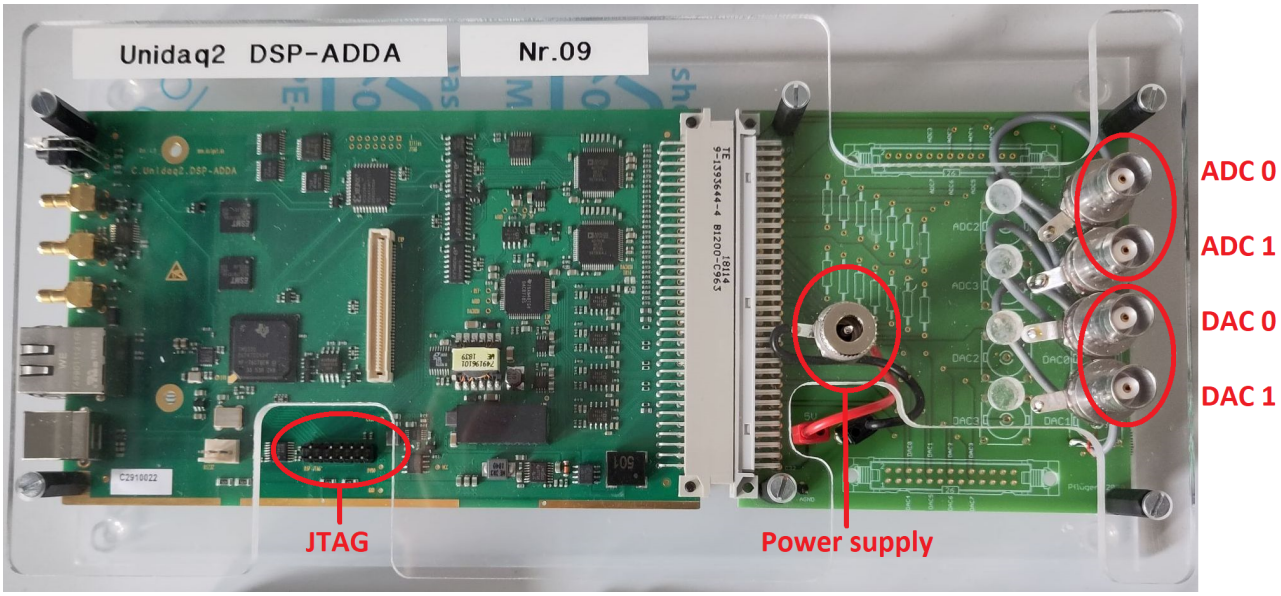


Figure 2.1.: Lab Hardware UniDAQ2

The inputs and outputs for the signals are labeled ADC 0/1 and DAC 0/1 in Fig. 2.1. The connection is made via BNC cable.

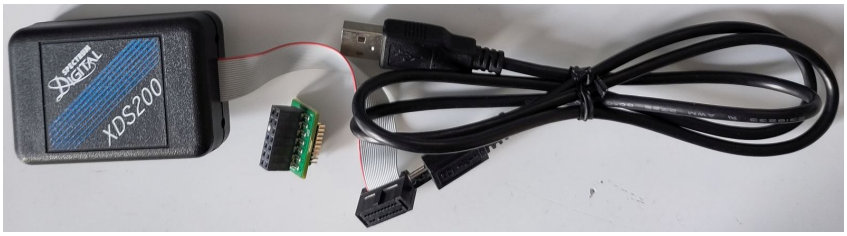


Figure 2.2.: JTAG emulator XDS200 with adapter and USB cable

2.2. Start of Code Composer Studio (CCS) and import of a project

A project with a simple echo program is available on the lab PCs. The source code can be found in the appendix A.2. Included are all necessary initializations and the prepared interrupt routines for ADC and DAC.
Hints:

- The present directory structure below D:\ti_work must be preserved at all times.
- To work with CCS, the project must always be stored locally on the PC.

Start CCS by double clicking the corresponding icon on the Desktop.



Open an existing project via ***Project→Import CCS Projects*** and choose the directory adda_simple in D:\ti_work\UniDAQ2.DSP-ADDA\Projects\bare (see Figure 2.3). Click ***Finish*** and you'll find the included files of the CCS project in the automatically opened Project Explorer.

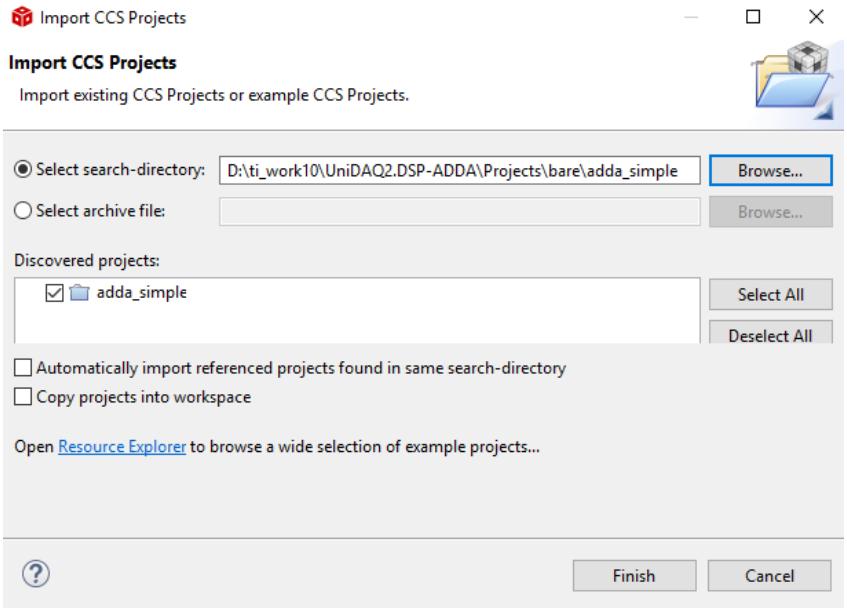


Figure 2.3.: Import a project

2.3. Compile, link and load the project

Clicking on the project in the Project Explorer marks it as active.
Select now ***Run*** → ***Debug*** ($< F11 >$). By this command

- the inserted files will be **compiled**,
- **linked**, while the linker is interpreting the file unidaq2.cmd
- and an **executable file** will be created.
- The target hardware (UniDAQ2 board) will be **initialised and connected**,
- The executable file *..\adda_simple\Debug\adda_simple.out* will be **loaded** to the board.
- The program will be **started** and **stopped at the row *void main(void)***.

The „Problems“ window should not show any error or warning and should not be opened, respectively.

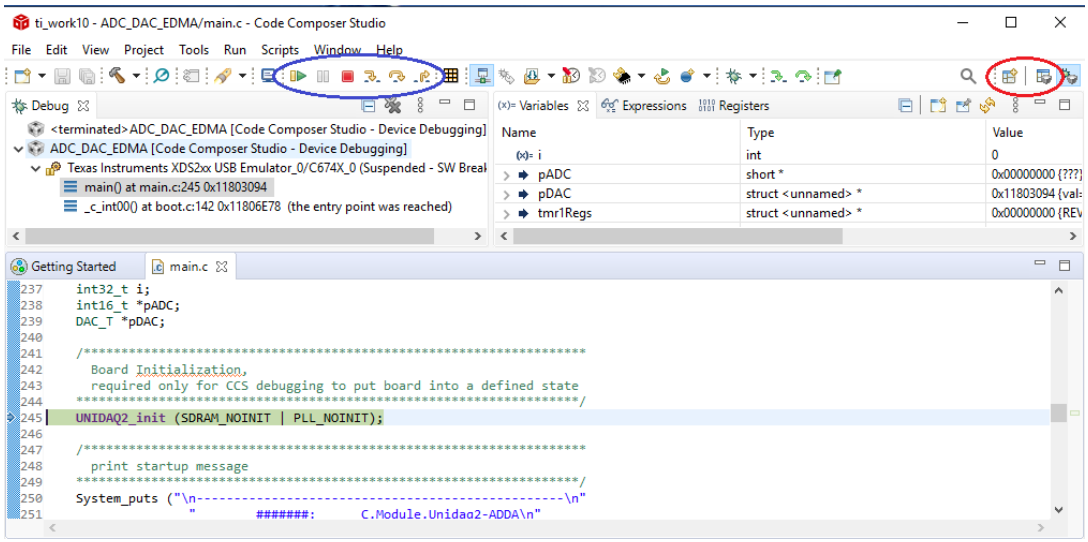


Figure 2.4.: View after Debug → Run

After that, you are automatically switched to the Debug Perspective. You can return to the CCS Edit-Perspective via the buttons on the right top (Figure 2.4, encircled in red).
Run → ***Resume*** or <F8> or the corresponding button (Figure 2.4, green arrow on the left of the area circled in blue) starts the program on the DSP.

2.4. Using the CCS Debugging tool

CCS offers excellent debugging possibilities, a selection of which is explained below. Refer back to it as needed during the lab.

2.4.1. Breakpoints

Double-click on the blue border to the left of the line number to set or remove a breakpoint. A breakpoint can also be placed in front of the line where the cursor is currently located via ***Run*** → ***Toggle Breakpoint***.
View → ***Breakpoints*** shows the existing breakpoints in a separate window in CCS, each of which can be activated or deactivated individually via checkboxes, see figure 2.5.
With the right mouse button on the breakpoint, then ***breakpoint properties*** opens a new window. At *Action* you can now define if the program should stop at the breakpoint (*Remain Halted*) or e.g. run again and again to this point via *Update View* or *Refresh All Windows* and update certain windows.

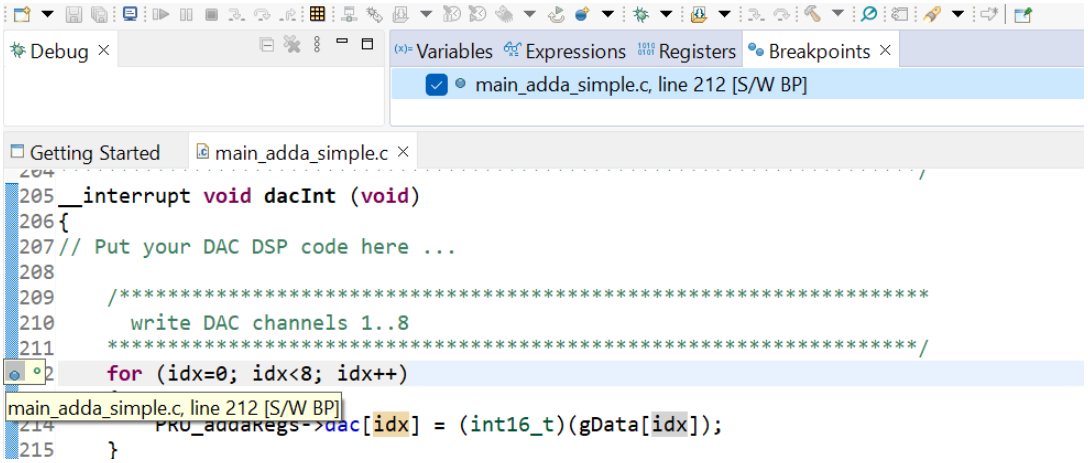


Figure 2.5.: Set breakpoint and breakpoint window

2.4.2. Display source and assembler code

View → ***Disassembly*** opens a window with the assembly code next to your C code, see Fig. 2.6. In this you can now monitor single steps. One or more (in case of parallel statements) blue arrows show the current position of the program counter.

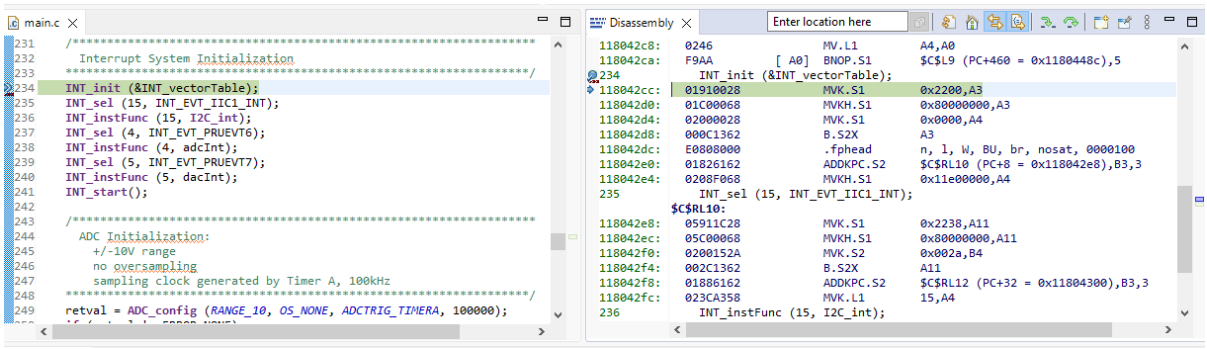


Figure 2.6.: View assembler code

2.4.3. Single step execution

By setting several breakpoints the program can be executed step by step from one breakpoint to the next.
But also by using the symbols marked in Fig. 2.7 the program can be executed in single steps, either in C-code (green circled symbols) or in assembler code (red circled symbols). The left symbol, "Step Into", also leads into the functions that are called; the right symbol, "Step Over", takes you one line forward in the code.

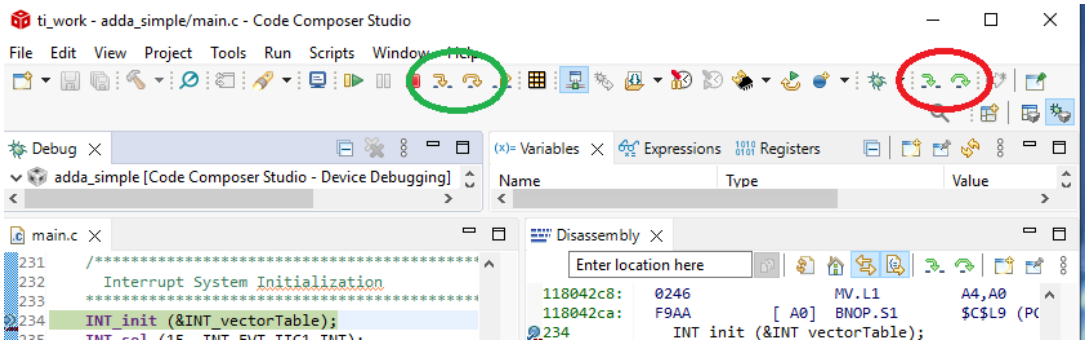


Figure 2.7.: Single step execution

2.4.4. Monitor and change variables

Via Expressions window
Monitor a variable by selecting it and choosing *Add Watch Expression...* after right mouse click. The Expressions window becomes active and the variable is displayed. Alternatively you can display the expressions window directly via ***View*** → ***Expressions*** and add a variable via *Add new expression*, see figure 2.8.
You can now assign a new value to the variables in the column *Value*.
Recently changed contents are highlighted in yellow.

Direct View
When you move the mouse pointer over the variable, its content (or the start address if it is a function) appears on the screen in a popup window.

2.4.5. Show DSP register contents

The registers can be monitored with ***View*** → ***Registers***, which opens a new window, see figure 2.9. Clicking on the arrow to the left of the entries (circled in red in the figure for the core registers) shows all the individual related registers.
Inside this window, the contents of the 32 A and B registers and the "program counter" PC can be monitored and changed under Core Registers (internal CPU registers). Other important registers are the interrupt registers ISTP, IFR, IER and IRP and the interrupt flag GIE (bit no. 0 in the CSR). In this window you also have access to the peripheral registers (for timer, interrupt and EMIF) and the registers of the serial interfaces.

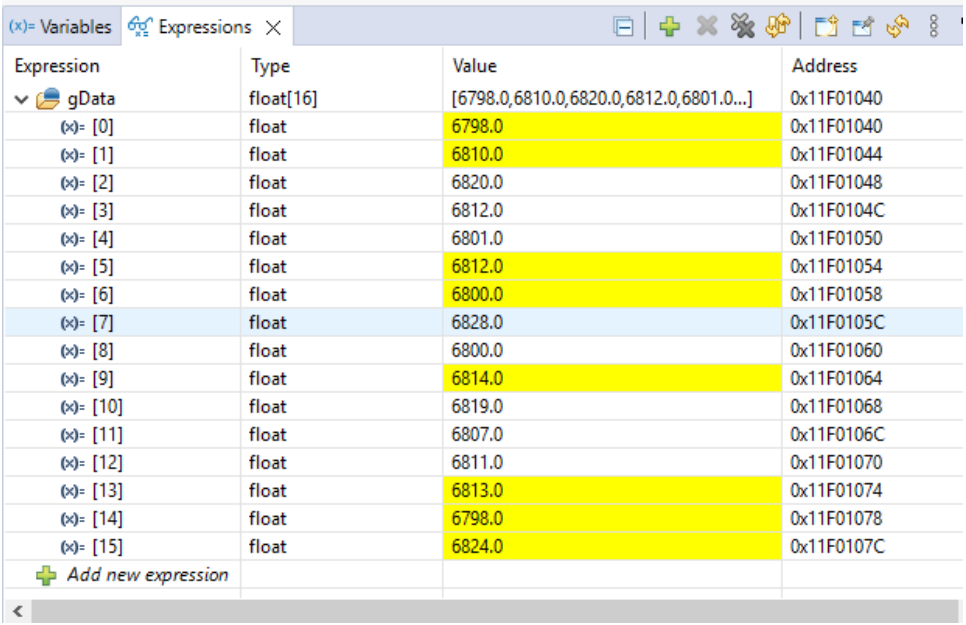


Figure 2.8.: Display variables

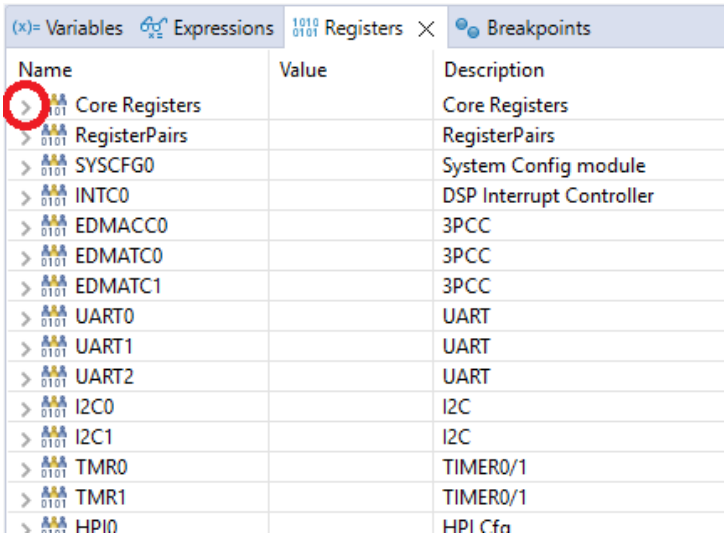


Figure 2.9.: Display register contents (detail)

2.4.6. Show memory content

Via ***View*** → ***Memory Browser*** a window is opened where you can view memory contents. There you can check how variables are stored by entering the address of a variable in the address field, e.g. &g_i16Data. If necessary the data format must be adapted, default is 32-bit hex.

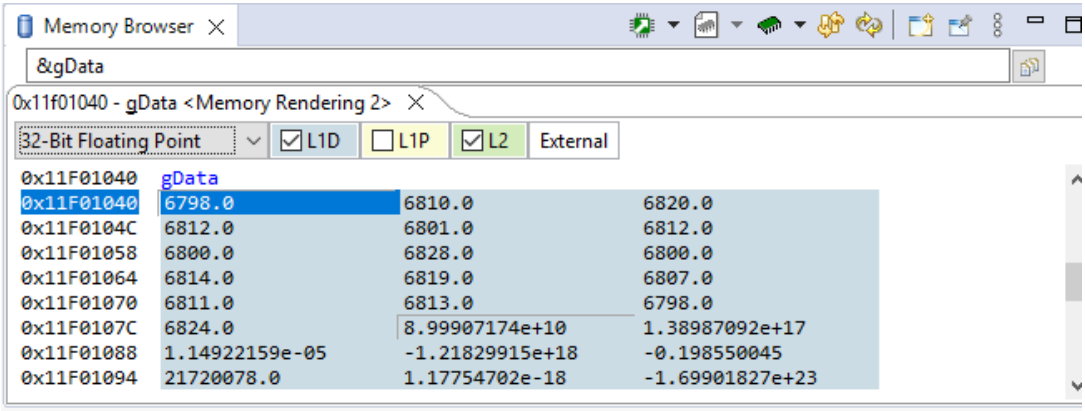


Figure 2.10.: Show memory contents

2.4.7. Show contents of arrays graphically (Graphical Display)

With CCS you can graphically display contents of data buffers (arrays in C code) in time or frequency domain. Thus this facility can be used like a built-in oscilloscope (see time domain, 2.4.7) or as a frequency analyzer (see frequency domain, 2.4.7).
So first the data to be displayed needs to be written into a buffer. The following example prepares the program for the representation of 1000 input values of the left and right channel resp. ADC 0 and 1:

- Extend your program with one or more global buffers for the values to be displayed according to the first lines of the listing 2.1.
- Implement lines 14-17 given in Listing 2.1 to store the current sample values in the buffers `g_i16InBuf_L` and `g_i16InBuf_R`. In this case the lines must be inserted into the ISR `interrupt void adcInt (void)`, *after* the assignment of the ADC register contents to `g_i16Data[]`. Now always the last read 1000 values are stored automatically.

For the representation of the processed values to be sent to the DAC, the same procedure can be followed.

Listing 2.1: Buffer_handling_ADC_DAC.c.

```
1  /* defines */
   #define LENGTH_ADC_BUF 1000
3  #define LENGTH_DAC_BUF 1000

5  /* Global variables */
   int16_t g_Count_ADC = 0;
7  int16_t g_i16InBuf_L[LENGTH_ADC_BUF];
   int16_t g_i16InBuf_R[LENGTH_ADC_BUF];
9  int16_t g_Count_DAC = 0;
   int16_t g_i16OutBuf_L[LENGTH_DAC_BUF];
11 int16_t g_i16OutBuf_R[LENGTH_DAC_BUF];

13 /* Insert in ISR adcInt */
       g_i16InBuf_L[g_Count_ADC] = g_i16Data[0];
15     g_i16InBuf_R[g_Count_ADC] = g_i16Data[1];
       if (++g_Count_ADC >= LENGTH_ADC_BUF)
17         g_Count_ADC = 0;

19 /* Insert in ISR dacInt */
       g_i16OutBuf_L[g_Count_DAC] = g_i16Data[0];
```

```
21      g_i16OutBuf_R[g_Count_DAC] = g_i16Data[1];
        if (++g_Count_DAC >= LENGTH_DAC_BUF)
23          g_Count_DAC = 0;
```

Time Domain

Via ***Tools*** → ***Graph*** → ***SingleTime*** resp. ***DualTime*** the configuration window opens initially, see Figure 2.11.
Enter the variable name in the "Start Address" field (different in the screenshot). For DualTime there are two, "Start Address A" and "Start Address B". Adjust the fields "Acquisition Buffer Size", "Dsp Data Type", "Sampling Rate" and "Display Data Size" - for the given example and a sampling rate of 100 kHz according to Fig. 2.11.
With Export/Import you can save your settings for later Graphical Displays and use them again.
Close the window with OK. In one or two new windows the values in the time domain are now displayed graphically, see Fig. 2.12.

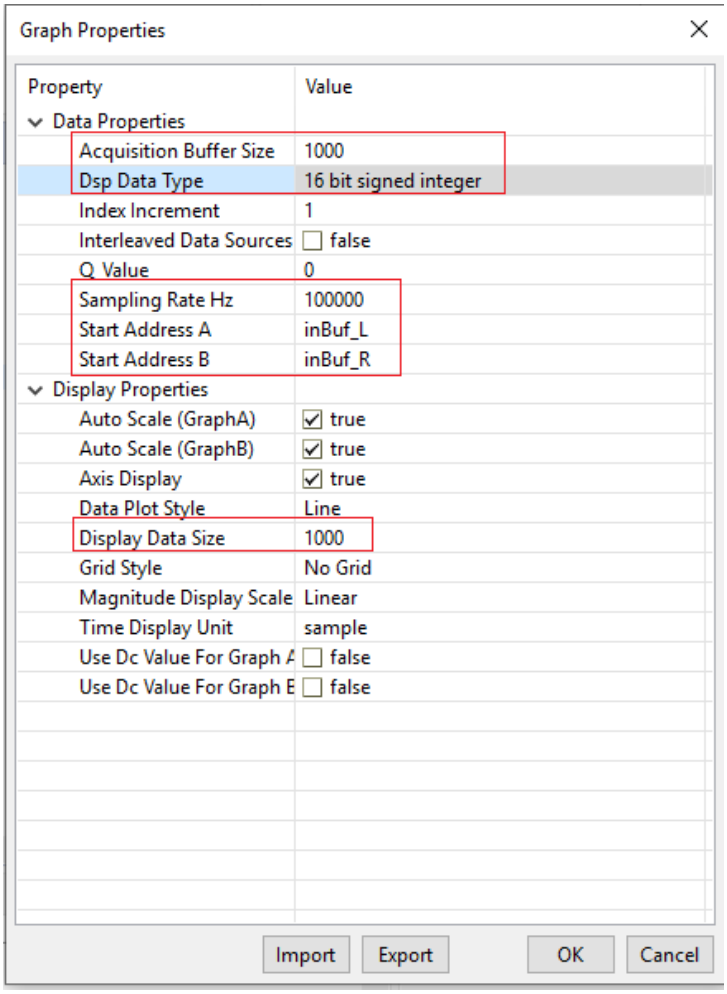


Figure 2.11.: Properties graph display

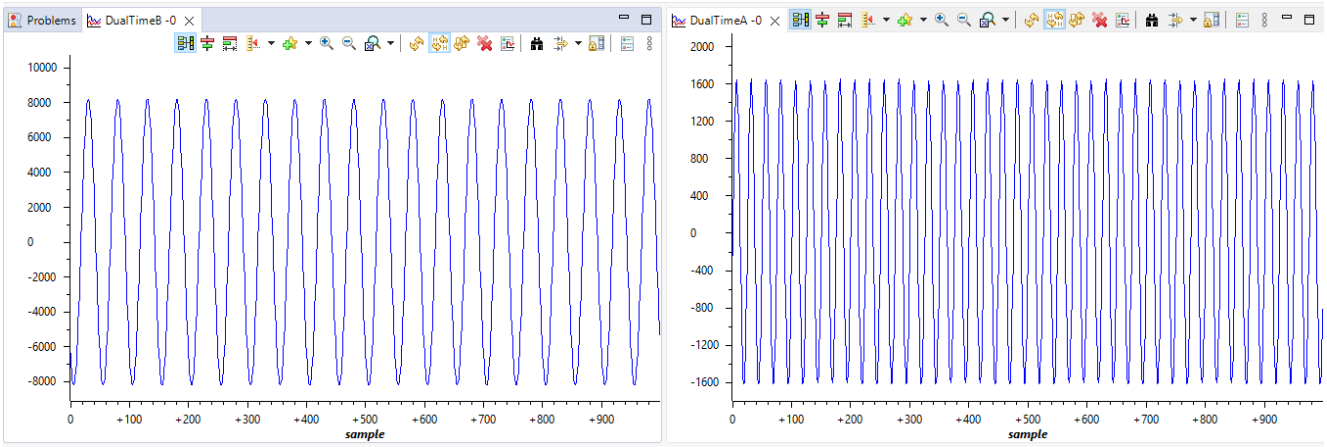


Figure 2.12.: Dual time graph. display

Frequency Domain

Tools → ***Graph*** → ***FFT Magnitude*** opens the configuration window for displaying the FFT of a buffer content, see left picture in Figure 2.13.
Set the ”FFT Order” to 8 (this will automatically set the ”FFT Frame Size” to 256). Adjust ”Acquisition Buffer Size”, ”Dsp Data Type” and ”Sampling Rate Hz”. ”Start Address” again is the buffers name. A new window will now display the FFT of the samples, an example can be seen on the right in Fig. 2.13.
Set ”Data Plot Style” to ”Line” if you want the values to be linked, or ”Bar” if you prefer them to be displayed as single vertical lines.

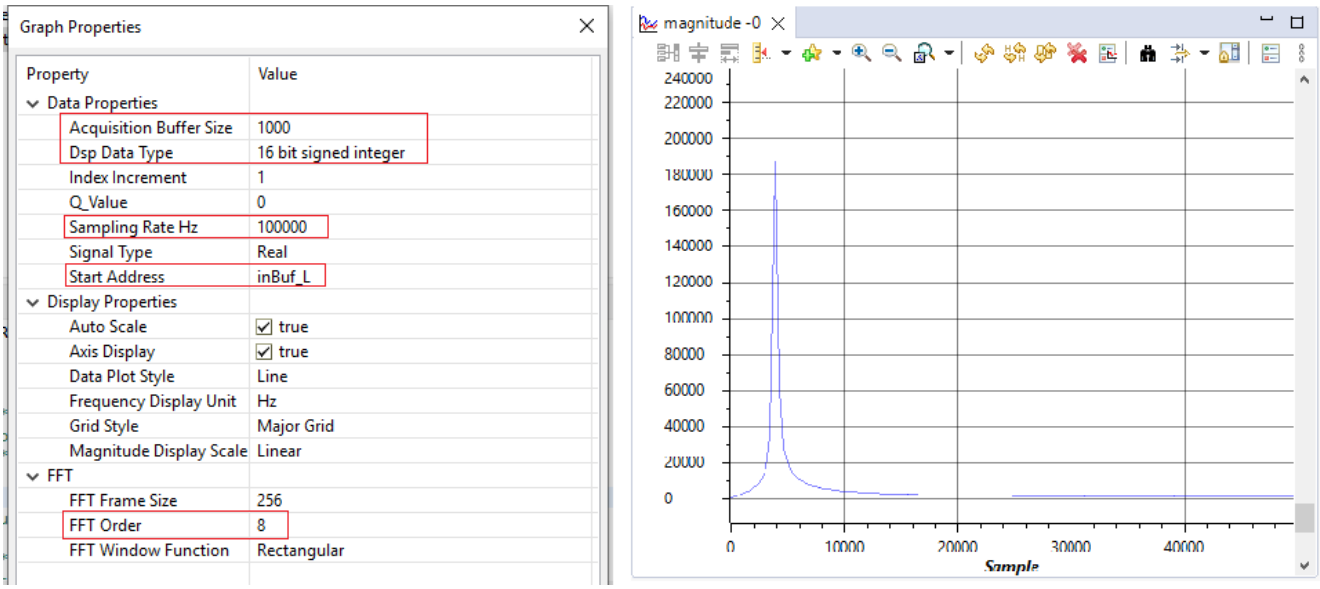


Figure 2.13.: Properties graph display and graph, FFT

Refreshing the Graph. Display

The windows are not refreshed immediately. If you want to have the refreshed buffer contents displayed while the DSP is running select *Refresh* after right mouse click.

Refresh the displays automatically to a *meaningful* time, e.g. when the buffer is completely refilled: For this, set a breakpoint at the position where the counter for reading in the data is set to zero in the if query, so that the breakpoint is always reached when the buffer is full. Set *Action* = *Refresh All Windows* or *Update View* in the breakpoint properties, for the latter select the graphics windows via *View* and start your program.
You can see the setting in Fig. 2.14.

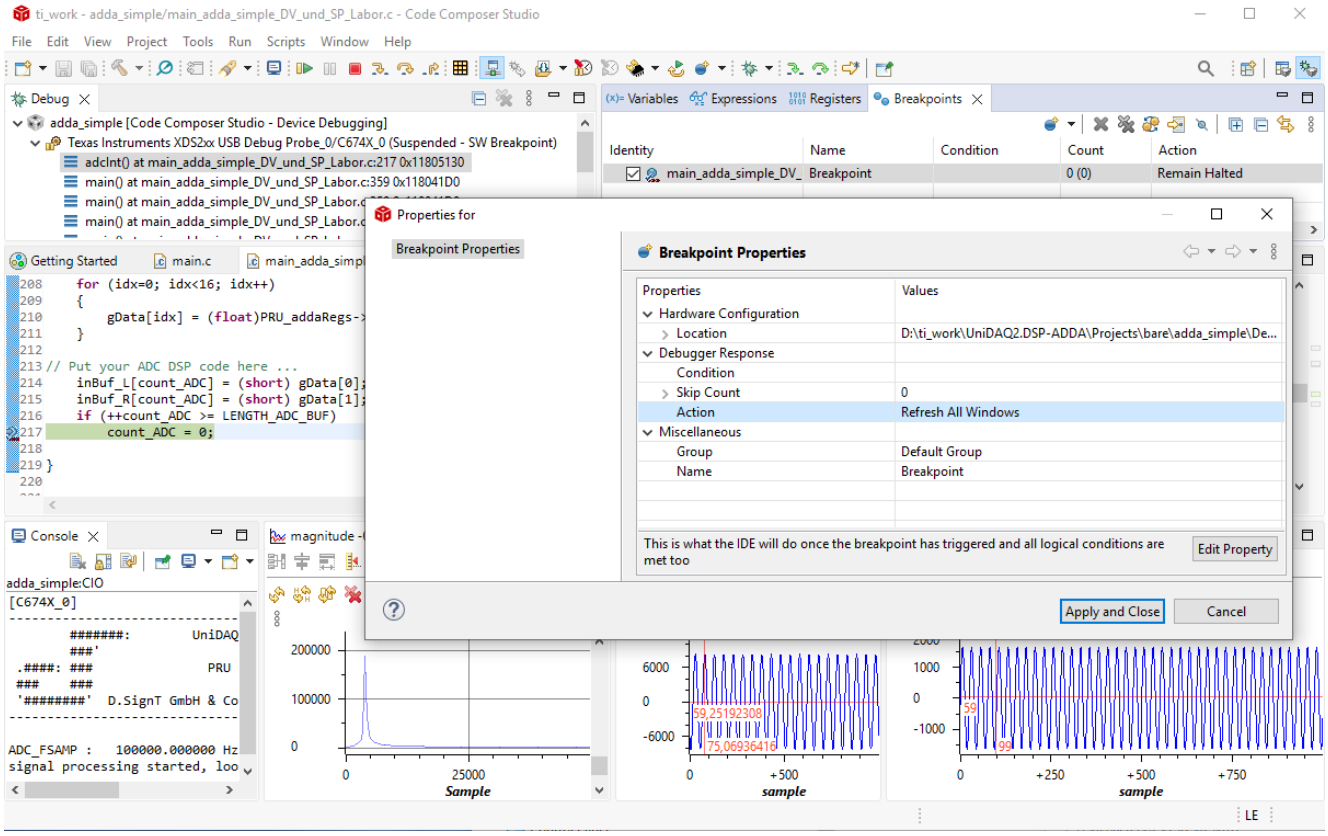


Figure 2.14.: Setting for automatic update

Hints for using the Graph. Display

- You can adjust the configuration (data details, display scaling) at any time using the button 
- You can change other properties of the display (colors, grid etc.) by selecting *Display Properties* after right-clicking the window.
- Pressing the left mouse button in the display causes a cursor cross which is moving with the mouse pointer, the values in the intersection point are displayed, see figure 2.15.

- You have to make screenshots with <ALT>+<Print> and paste them into e.g. MS Paint with <Ctrl>+<v> - or use Windows' Snipping Tool.
- Please note that the display on the oscilloscope no longer provides meaningful results in breakpoint mode.

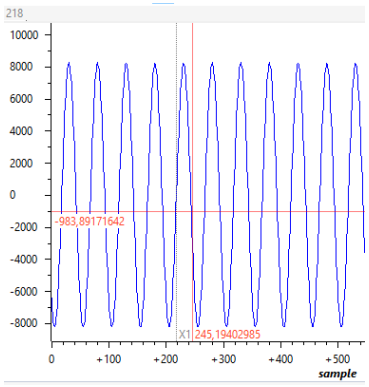


Figure 2.15.: Cursor in Graph. Display

2.4.8. Code Profiling

Profiling can be used, for example, to determine how many clock cycles are required for the execution of code.
Only this functionality is described here.

1. Load your program via ***Run*** → ***Debug***.
2. Switch on the Profile Clock with ***Run*** → ***Clock*** → ***Enable***. A clock symbol appears on the bottom of the CCS window, see figure 2.16.
3. Set a breakpoint directly in front of the code to be analysed. Run the program up to this point.
4. Then call ***Run*** → ***Clock*** → ***Reset*** (or doubleclick the clock symbol to reset the counter).
5. A second breakpoint must now be set after the code to be analysed.
6. Starting the program, the clock cycles will be counted until the program stops again.

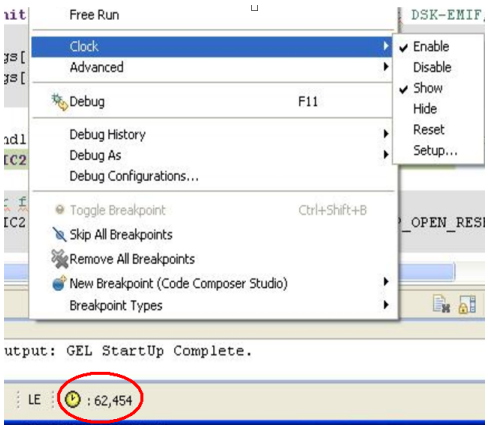


Figure 2.16.: Profile Clock, clock cycles

2.5. Signal input and output

2.5.1. Signal input

Input signals can be fed from a generator (UPV or function generator) to the BNC input connectors ADC 0 and ADC 1 (see fig. 2.1). For instructions on how to use UPV, see UPV Introduction [2].

Maximum input voltage: $5\ V_{pp}$

For feeding an audio signal from the sound card of the PC an adapter 3,5mm jack → BNC is needed. Use Audacity to play wavefiles, e.g. from D:\wavefiles. Switch on loop mode to avoid having to start playback repeatedly: red circle in fig. 2.17.

The read-in data are made available in the C program in the ISR `interrupt void adcInt (void)` in the `int16_t` variables `g_i16Data[0]` and `g_i16Data[1]`.

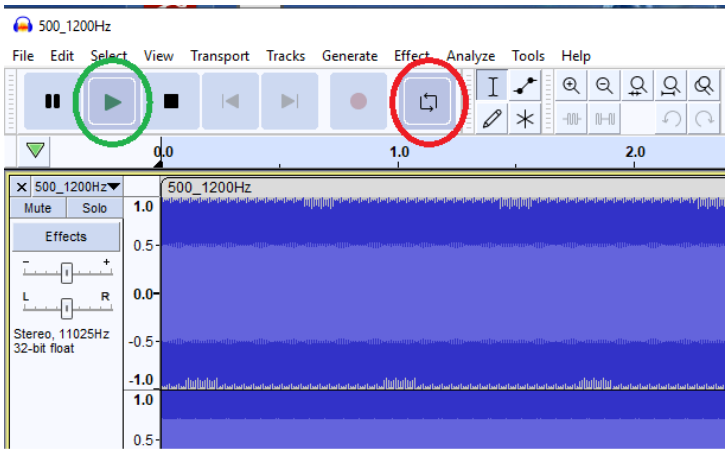


Figure 2.17.: Audio playback using Audacity

2.5.2. Signal output

Data to be output must be transferred to the `int16_t` variables `g_i16Data[0]` and `g_i16Data[1]`. In the ISR `interrupt void dacInt (void)` these are sent to the DAC.

The output signal of the board is available at the sockets DAC 0 and DAC 1, these can be connected to the audio analyzer (UPV) or oscilloscope.
For a playback of the signal via loudspeakers an adapter BNC → 3,5mm jack is needed here as well.



Chapter 3

Sampling Rates

3.1. Setting the sample rates

The preset sample rate in main_adda_simple.c is $F_s = 100\ kHz$ for ADC and DAC respectively. It is possible to reduce the 100 kHz via **integer divisors ADC_DIV and DAC_DIV**, see listing 3.1. If for example 3 is selected for ADC_DIV and DAC_DIV, the values are read in and output via ADC and DAC with 33.333 kHz each.

Listing 3.1: Definition Samplerates

```
1 // default ratio of sample rates for ADC and DAC
  #define ADC_DIV 2
3 #define DAC_DIV 2

5 #define ADC_FSAMP (100000/ADC_DIV)
  #define DAC_FSAMP (100000/DAC_DIV)
```

The actual frequencies of the ADC and DAC are output when the program is started.

3.2. Operation with different sampling rates on ADC and DAC

For multirate systems it is necessary to operate ADC and DAC with different frequency. Following the pattern described in section 3.1 different dividers ADC_DIV for the ADC and DAC_DIV for the DAC can be defined for this purpose. The interrupt routines are each called with the selected sampling frequency.

Chapter 4
Characteristics of the UniDAQ2 board

4.1. Amplitude of the UniDAQ2

The UniDAQ2 board presents the transmission response shown in Fig. 4.1 at a sampling rate of 100 kHz.
This characteristic is overlaid on any amplitude response intended by an implementation on the board.

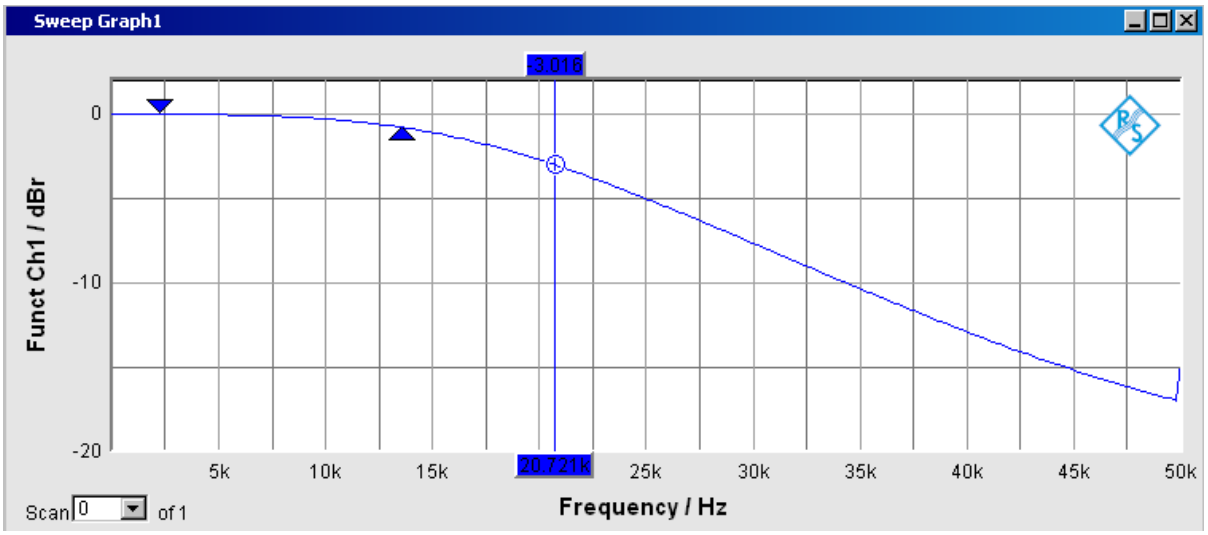


Figure 4.1.: Amplitude response UniDAQ2

4.2. Anti-aliasing filtering

The hardware does **not** have a built-in anti-alias filter. Accordingly, interferences are measurable. For an exemplary representation of the interferences, a 10 kHz signal was fed in and the output of the board was connected on the one hand directly and on the other hand via a subsequent lowpass

filter (Kemo) to the audio analyzer UPV.
In Fig. 4.2 the output signal of the board is shown in the time domain on channel 1 (blue), on channel 2 (red) one sees the subsequently filtered signal. Already in the time domain it can be seen that the unfiltered signal has interferences.

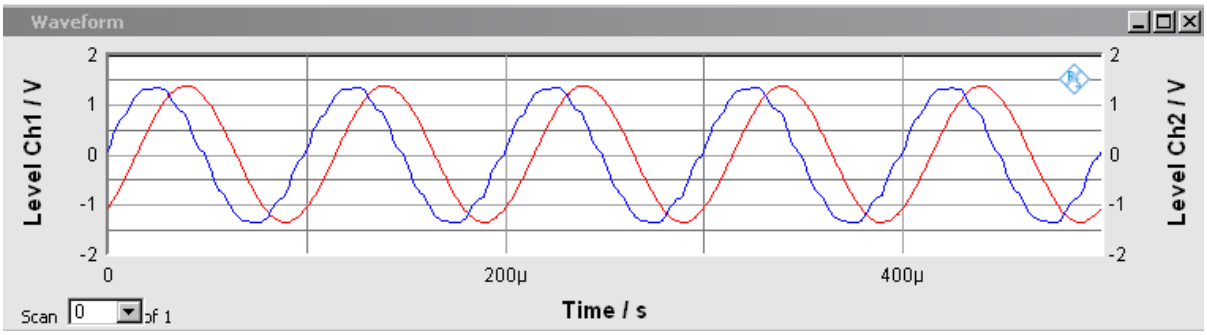


Figure 4.2.: Output signal without and with subsequent filtering

Fig. 4.3 shows the spectrum without subsequent Kemo filter, in which one can clearly see the mirror components above $F_s/2$ and especially the component at 90 kHz with a level of -30 dBr. In Fig. 4.4 the spectrum after filtering is shown.

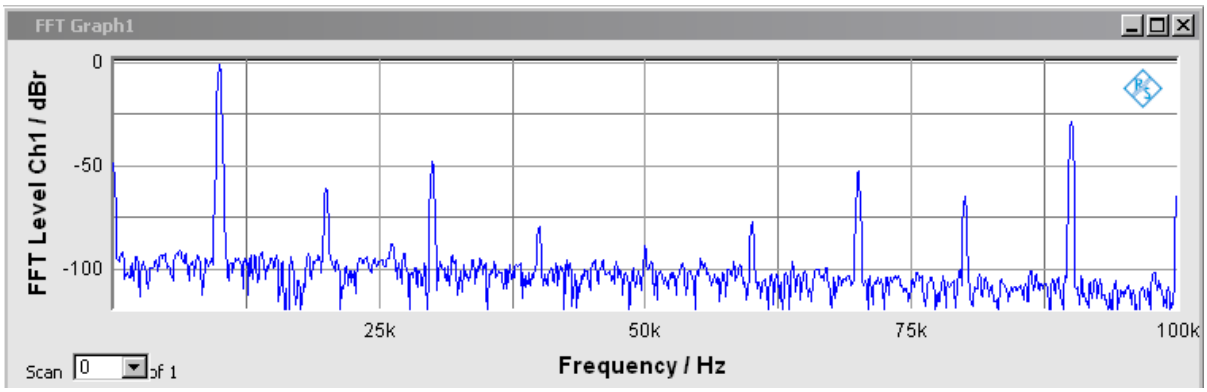


Figure 4.3.: Spectrum of the output signal without subsequent filtering

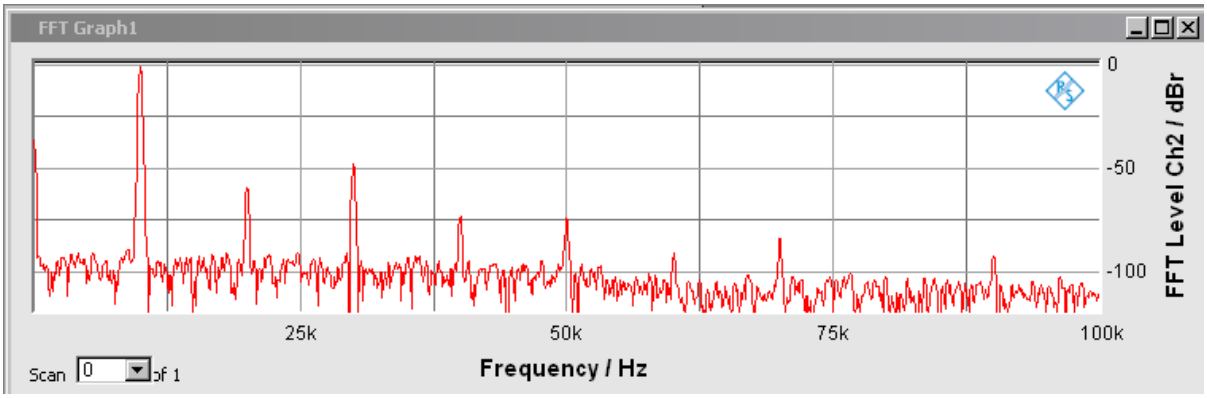


Figure 4.4.: Spectrum of the output signal with subsequent filtering

Therefore, to suppress frequencies greater than $F_s/2$, a low-pass filter should always be connected in series at the output, the cutoff frequency of which must be set to $F_s/2$. The lab workplaces each have a suitable Kemo filter for this purpose, the use of which is described in section 4.2.1.

4.2.1. Kemo Filter

Each device *Kemo BM 8* contains two separately usable filters, so that one filter can be used at each workstation. The input socket is on the left in each case, the output of the filter on the right. For separated use of the filters, the switch in the middle between the two must be on the left switch position "Sep".
Details of the device can be found in the data sheet, see bibliography, 3.
Fig. 4.5 shows exemplary the setting of the device for a low pass filter with a cutoff frequency of 50 kHz: **5 . 0 0 x LP** 10^4



Figure 4.5.: Kemo filter setting

Appendix A

Appendix: UniDAQ2

A.1. Short presentation UniDAQ2

The UniDAQ2 [1] is a versatile data acquisition and signal processing system for industrial applications, scientific research and education.
16 simultaneously sampled analog inputs with excellent performance and low latency qualify the UniDAQ2 for high accuracy dynamic signal measurement and control. 8 analog outputs control actuators, generate stimuli, reference voltages or can be used to monitor internal processing states.

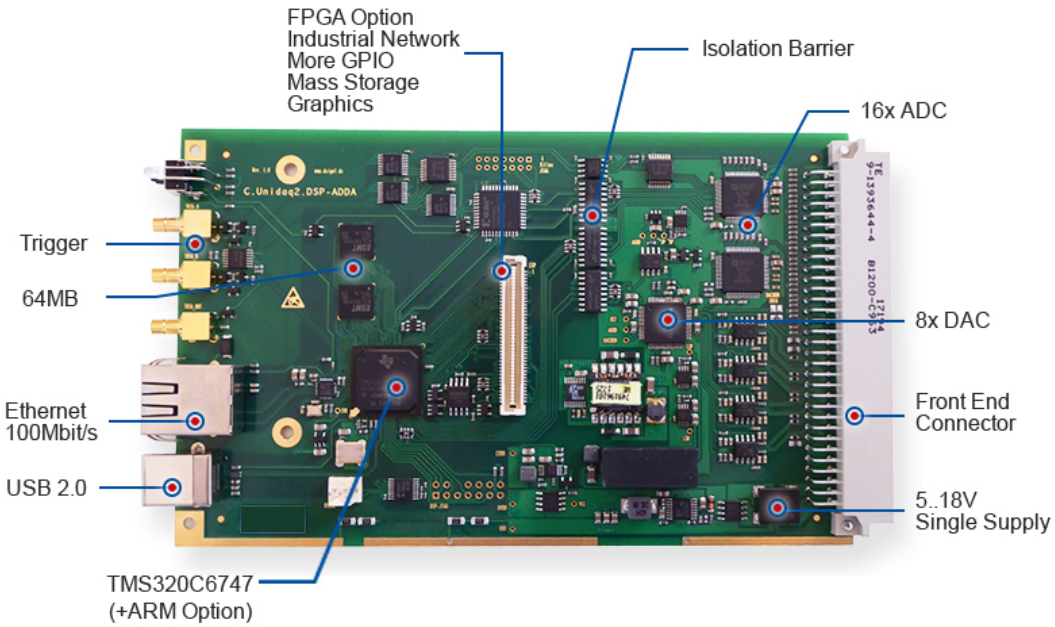


Figure A.1.: UniDAQ2_DSP-ADDA

A.1.1. Typical fields of application

- Oscillation analysis and machine monitoring
- acoustic signal evaluation, microphone array measurement technology
- Regulators and positioning systems
- Measuring systems for research, development and test benches
- Laboratory equipment for training in signal processing and mechatronics

A.1.2. Software

- Web frontend for quick access to operation, configuration and visualization of your data
- Direct access via Matlab® Instrument Control Toolbox, Python or C programs on the PC through TCP/IP protocol interface
- DSP library to support your device configuration, data acquisition, and communications
- Driver for TCP/IP, USB-CDC, USB-TMC and USB-DFU

A.1.3. Key Features

- 16 simultaneously sampling SAR A/D converters, 16-bit, 175kS/s
- 8 inherently monotonic D/A converters, 16-bit, 105kS/s
- internal or external sample clock generation for ADC and DAC.
- 456 MHz floating point DSP (TMS320C6747) with 64 Mbyte SDRAM
- USB2.0 device port and 100Base-T Fast Ethernet
- Expansion port for mass storage, fieldbus connection, display, GPIOs
- isolated analog section
- only one power supply 5-18V, approx. 3W

A.2. Basic Sourcecode

The following source code is available in a prepared CCS project in the lab.

Listing A.1: main_adda_simple_lab.c.

```
/*****************************************************************

 @file      main_adda_simple_Lab.c (Svg, 11-Jul-2023)

 @brief     C.Module.Unidaq2-ADDA Test Program
 @author    D.SignT GmbH & Co. KG, A.Klemenz, modified for MSVC simulation by
     Svg
 @version   1.0
 @date      2018-01-31, Svg; 2025-09-30, Ltl,Kpke
 @target    C.Module.Unidaq2-ADDA
 @compiler  CCS
 @history   1.0 initial release 2018-01-31 by AK                    */

/*****************************************************************
  includes
 *****************************************************************/
#include <BoardSupport/inc/unidaq2.h>       /**< Unidaq2-ADDA BIOS    */
#include <BoardSupport/inc/errors.h>        /**< Error Codes          */
#include <BoardSupport/drv/i2c.h>          /**< Unidaq2-ADDA I2C     */
#include <BoardSupport/drv/adda.h>          /**< Unidaq2-ADDA ADC DAC */

#include <Common/pru.h>                     /**< PRU subsystem      */

#pragma DATA_SECTION (PRU_addaCode, ".const_slow");
#include <Common/PRU/PRU_adda_bin.h>        /**< PRU program code   */

#include <ti/pspiom/cslr/soc_C6747.h>         /**< 6747 peripherals   */
#include <ti/pspiom/cslr/cslr_gpio.h>         /**< GPIO               */

#include <math.h>                     /**< floating-point     */


/*****************************************************************
  defines
 *****************************************************************/
// default ratio of sample rates for ADC and DAC
#define ADC_DIV 2
#define DAC_DIV 2

#define ADC_FSAMP (100000/ADC_DIV)
#define DAC_FSAMP (100000/DAC_DIV)

//#define SYSMSG_DBG_UART     /**< system messages via Debug UART       */
#define SYSMSG_CCS_CIO      /**< system messages via CCS CIO          */

#ifdef SYSMSG_CCS_CIO
#include <stdio.h>
#endif

// PRU
#define  PRU_NUM     0        /**< the PRU to handle McASP (0 or 1) */


/*****************************************************************
  globals (don't touch)
```

```
 ***************************************************************************/
/*
  Global variables used in interrupts must be declared volatile.
  Place in L1DRAM (.data_crit section) to avoid cache data
  displacement and fastest access time.
 */
#pragma DATA_SECTION (g_i16Data, ".data_crit");

volatile int16_t g_i16Data[2];

PRU_addaOvly PRU_addaRegs = (PRU_addaOvly) (PRU0_DRAM + PRU_NUM * (PRU1_DRAM-
    PRU0_DRAM));

/***************************************************************************
  globals SP Lab
 ***************************************************************************/

// Put your additional global variables here ...



/***************************************************************************
  end of globals SP Lab
 ***************************************************************************/

/***************************************************************************//**
  @brief system message output on Debug UART or via Code Composer
  @param    pstr - pointer to string
  @return   -
 ***************************************************************************/
void System_puts (char *pstr)
{
#ifdef SYSMSG_DBG_UART
    /*************************************************************************
      locals
     *************************************************************************/
    char c;

    /*************************************************************************
      write to Debug UART
     *************************************************************************/
    while ((c = *pstr++) != 0)
    {
        while (DBG_write(c) != DBG_OK) ;
    }
#endif

#ifdef SYSMSG_CCS_CIO
    /*************************************************************************
      write to CCS CIO
     *************************************************************************/
    printf (pstr);
    fflush(stdout);
#endif
}

/***************************************************************************
  @brief    PRU interrupt for ADC
  @param    -
  @return   -
```

```
 ***************************************************************************/
__interrupt void adcInt (void)
{
    //  read ADC channels
    g_i16Data[0] = PRU_addaRegs->adc[0];
    g_i16Data[1] = PRU_addaRegs->adc[1];

    // Put your ADC DSP code here ...


}

/*****************************************************************************//**
  @brief    PRU interrupt for DAC
  @param    -
  @return   -
 ***************************************************************************/
__interrupt void dacInt (void)
{

    // Put your DAC DSP code here ...


    // write DAC channels
    PRU_addaRegs->dac[0] = g_i16Data[0];
    PRU_addaRegs->dac[1] = g_i16Data[1];
}

/*****************************************************************************//**
  main program
 ***************************************************************************/
void main (void)
{
    // Put your variables to be initialized in main() here ...

    /*************************************************************************
     locals
     ************************************************************************/
    //    CSL_GpioRegsOvly GpioRegs = (CSL_GpioRegsOvly)CSL_GPIO_0_REGS;
    int32_t retval;

    /*************************************************************************
     Board Initialization,
     required only for CCS debugging to put board into a defined state
     ************************************************************************/
    UNIDAQ2_init (SDRAM_NOINIT | PLL_NOINIT);

    /*************************************************************************
     print startup message
     ************************************************************************/
    System_puts ("\r\n-------------------------------------------------\r\n"
            "         ########:         UniDAQ2.DSP-ADDA\r\n"
            "         ###'\r\n"
            "  .####:  ###                PRU ADDA Demo\r\n"
            "  ###     ###\r\n"
            "  '########'   D.SignT GmbH & Co.KG   www.dsignt.de\r\n"
            "-------------------------------------------------\r\n\n");
    printf("ADC_FSAMP : %15.6f Hz, DAC_FSAMP : %15.6f Hz\n", (float) ADC_FSAMP,
        (float) DAC_FSAMP);
```

```
176     /**********************************************************************
          I2C Initialization
178      **********************************************************************/
        retval = I2C_init (400000, 0x7E);
180     if (retval != ERROR_NONE)
        {
182         System_puts("I2C_init() failed! program halted\r\n");
            for (;;);  /* trap or error handling */
184     }

186     /**********************************************************************
          Interrupt System Initialization
188      **********************************************************************/
        INT_init (&INT_vectorTable);
190     INT_sel (15, INT_EVT_IIC1_INT);
        INT_instFunc (15, I2C_int);
192     INT_sel (4, INT_EVT_PRUEVT6);
        INT_instFunc (4, adcInt);
194     INT_sel (5, INT_EVT_PRUEVT7);
        INT_instFunc (5, dacInt);
196     INT_start();

198     /**********************************************************************
          ADC Initialization:
200         +/-10V range
            no oversampling
202         sampling clock generated by Timer A, 100kHz
         **********************************************************************/
204     retval = ADC_config(RANGE_10, OS_NONE, ADCTRIG_TIMERA, ADC_FSAMP);  // Svg
        if (retval != ERROR_NONE)
206     {
            System_puts("ACD_config() failed! program halted\r\n");
208         for (;;);  /* trap or error handling */
        }
210
        /**********************************************************************
212      DAC Initialization:
           +/-10V range
214        synchronized to ADC
           no calibration
216        no monitor output
         **********************************************************************/
218     retval = DAC_config(DAC_CFGREG_GAIN4, DACTRIG_TIMERB, DAC_FSAMP, NULL, NULL
            , REF5V, MON_OFF);

220     if (retval != ERROR_NONE)
        {
222         System_puts("DAC_config() failed! program halted\r\n");
            for (;;);  /* trap or error handling */
224     }

226     /**********************************************************************
         enable PRU, load and execute PRU code
228      **********************************************************************/
        PRU_init();
230     PRU_load(PRU_NUM, (void *)PRU_addaCode, sizeof(PRU_addaCode));
        if (PRU_run(PRU_NUM) != ERROR_NONE)
232     {
            System_puts("PRU_run() failed! program halted\r\n");
234         for (;;);  /* trap or error handling */
```

```
    }

    /*********************************************************************
      start data acquisition
     *********************************************************************/
    System_puts("signal␣processing␣started,␣loopback␣ADC␣1..8␣->␣DAC␣1..8\r\n")
        ;
    DAC_start();
    ADC_start();

    /*********************************************************************
     main program loop
     *********************************************************************/
    for (;;)
    {
        // infinite for loop
    } // for (;;)...
}
```

Bibliography

[1] DSignT: *UniDAQ Processor Board UniDAQ2.DSP-ADDA*.
https://www.dsignt.de/de/unidaq/unidaq2-dsp-adda.html

[2] UPV_Starter_en.pdf: *Introduction Audioanalyser R&S UPV*.
moodle Course of the lab

[3] Datasheet-BM8-May-2021.pdf: *Datasheet Kemo BM 8*.
moodle Course of the lab